

Pathfinding Model and Lagrangian-Based Global Routing

Pengju Yao, Ping Zhang, Wenxing Zhu

Center for Discrete Mathematics and Theoretical Computer Science, Fuzhou University, Fuzhou, 350116, China
{215420017, 211027184, wxzhu}@fzu.edu.cn

Abstract—Global routing is a critical step in VLSI physical design. This paper proposes a novel pathfinding model based on integer linear programming for VLSI global routing. The Lagrangian relaxation method combined with a direction-aware weighted A*-algorithm is developed to quickly solve the model to obtain a better initial routing solution, which is further optimized by a designed multi-stage rip-up & rerouting algorithm. In each stage of rip-up & rerouting, different routing algorithms and cost functions are used to optimize the overflow and wire length. SPRoute and CUGR are two state-of-the-art global routers. Our proposed global routing algorithm outperforms SPRoute 1.0 & 2.0 in both the wire length and the number of vias on the ISPD08 benchmarks. On the ISPD18 benchmarks, compared to CUGR, our algorithm has about 5.1% reduction in the number of vias, and the average runtime is $4.89\times$ speedup; compared to SPRoute 2.0, our algorithm has about 1.7% reduction on the average wire length, and the number of vias is comparable.

Index Terms—Global Routing, Integer Linear Programming, Lagrangian relaxation, Rip-up & Rerouting

I. INTRODUCTION

With the rapid development of modern integrated circuit design, the design scale has increased dramatically which increases the complexity of each physical design stage largely. Routing is a critical step in VLSI physical design, needing to deal with millions of nets. Since the routing problem is NP-hard, its inherent complexity makes it difficult to obtain accurate and good results. Hence, designers usually decompose the routing problem into two stages: global routing and detailed routing.

Global routing divides a routing area into a grid map, on which the nets are roughly routed by minimizing the total wire length, overflow, and the number of vias. Detailed routing assigns the wires of nets to routing tracks according to the global routing result. Global routing is very important in physical design, since it affects detailed routing results, which in turn affects chip performance greatly.

Current global routing algorithms can be roughly divided into two categories: sequential global routing based on rip-up & rerouting [1]–[4] and concurrent global routing based on integer linear programming (ILP) [5]–[7]. Sequential global routing algorithms route nets one by one in sequence, and are efficient. Concurrent global routing algorithms usually solve an ILP model using commercial solver to get high quality routing results, which may take long runtime.

This paper attempts to combine the advantages of the two classes of algorithms, by proposing a novel mathematical model for VLSI global routing. The model can be handled quickly and comprehensively using a proposed algorithm for the corresponding Lagrangian relaxation problem and the subsequent multi-stage rip-up and rerouting algorithm.

A. Related Works

Due to the large scale, VLSI global routing is usually divided into two stages: initial routing stage and rip-up & rerouting stage. Initial routing completes routing of all nets to provide an initial routing result, which is improved by the subsequent rip-up & rerouting.

Sequential global routing algorithms NTHU-Route 2.0 [8] and SPRoute [3] use pattern routing for an initial routing result, while NCTU-GR 2.0 [1] uses monotonic routing. All of them can quickly obtain an initial routing solution, but the congestion may be large. Recently, CUGR [4] proposes 3D pattern routing to reduce overflow, but will take more runtime than 2D initial routing.

This work was supported by the National Natural Science Foundation of China under Grant 62174033.

Whereas, concurrent global routing algorithms solve an ILP model to obtain an initial routing result with less or without congestion. Generally, limited routing pattern candidates for each net must be generated prior for establishing their ILP models. BoxRouter 2.0 [5] and Sidewinder [9] propose a maximum routable ILP model, which routes as many nets as possible without congestion by using several routing patterns. Due to limited routing patterns for each net, the two routers may cause some nets disconnected, requiring post-processing to produce a legal final result. While, GRIP [6] proposes an ILP formulation that minimizes the total wire length and the number of vias, which includes many routing patterns. For their ILP, the LP relaxation is restricted to a small number of routing patterns and is solved by the column generation method, and then the obtained solution is optimized using a local improvement procedure to consider other patterns. Since the restricted LP relaxation is solved iteratively, GRIP [6] is slow. In addition, BonnRoute [7] and Ref. [10] propose routing models based on multi-commodity flow, in which multiple minimum rectilinear Steiner trees must be generated for each net in advance as routing pattern candidates, which is an NP-hard problem. Generally, the quality of concurrent global routing solution depends on the number of candidate routing patterns.

When solving their ILP models for global routing, BoxRouter 2.0 [5] and GRIP [6] first delineate a routing bounding box or decompose the routing region, and then use commercial solver to solve their ILP problems in a specific subregion. BFG-R [11] develops a discrete Lagrange multiplier method for 3D global routing with ILP formulation. Essentially, the algorithm of BFG-R is sequential, since once a net is routed, the multipliers are immediately updated heuristically for guiding the routing of next net. Although the two kinds of methods produce good solution quality, they take a lot of runtime and even cannot solve the problem if the routing scale is too large.

In the rip-up & rerouting stage, concurrent global routing algorithms use some post-processing techniques to optimize the initial routing results. While, sequential global routing algorithms require complex processing techniques. To relieve routing congestions, NCTU-GR 2.0 [1] and FastRoute 4.0 [2] use different cost functions and different routing orders combined with maze routing in 2D region. MGR [12] and CUGR [4] use multi-level framework combined with 3D maze routing to reduce overflow. Note that, the rip-up and rerouting algorithms of these academic global routers are in one or two stages, and each stage simultaneously optimizes wire length and congestions. They may get stuck at local optima as the number of iterations increases [13], and thus may cost a lot of runtime for obtaining a desirable global routing solution.

B. Our Contributions

By considering the pros and cons of concurrent and sequential global routing algorithms, this paper develops a novel ILP based pathfinding model and proposes a global routing algorithm for the VLSI global routing problem. The main contributions are summarized as follows:

- We develop a novel ILP based pathfinding model for VLSI global routing, which minimizes wire length subject to congestion constraints, but does not need to generate candidate routing patterns of nets prior.
- We propose a Lagrangian relaxation method combined with a gradient ascent method to update the multipliers, in which a

direction-aware weighted A*-algorithm is used to quickly solve a subproblem. The A*-algorithm searches more regions to reduce congestion, while avoids generating excessive vias. Moreover, the proposed method generates initial routing with congestion less than the pattern routing used in SPRoute [3].

- We propose a multi-stage rip-up & rerouting algorithm to optimize the initial routing result, in which each stage uses different routing algorithms and cost functions. It can effectively jump out of local minima on hard-to-route benchmarks to improve the solution quality and speed.
- Experimental results on the ISPD08 benchmarks show that our algorithm produces solutions with shorter wire length and smaller number of vias, compared with SPRoute 1.0 and SPRoute 2.0. On the ISPD2018 benchmarks, our algorithm is also efficient and effective, compared with CUGR and SPRoute 2.0.

The rest of this paper is organized as follows: Section II introduces the global routing problem and the flow of our global routing algorithm. Section III details the key techniques and algorithms of our global routing. Experimental results are presented in Section IV, and Section V concludes this paper.

II. PRELIMINARIES

In this section, we introduce the global routing problem and the flow of our global routing algorithm.

A. Overview of Global Routing Problem

In global routing, a die is usually divided into a collection of square cells called G-Cells. Then, a grid graph is defined as an undirected graph $G(V, E)$, where the nodes $v \in V$ denote the routing grid cells, and the edges $e \in E$ represent the connections of adjacent grid cell pairs. Each edge e is in the same direction of its belonged metal layer, either in the horizontal direction or in the vertical direction. Moreover, adjacent metal layers are connected by vias. Figure 1(a) gives an example of 3×3 grid, and Figure 1(b) shows a three-layer grid graph of a circuit containing 3×3 G-Cells in each metal layer. Moreover, each edge $e(u, v)$ has a capacity, that is, the maximum number of wires that can be routed between vertices u and v . The demand of an edge, $d(e)$, is the amount of wires passing through the edge e , and the overflow of the edge represents the amount of demand that exceeds the capacity.

The global routing problem can be described as: given a grid graph and a set of nets to be routed, find a routing scheme for each net such that all pins of the net are connected, meanwhile minimize the sum of the overflows of all edges. In addition, minimizing the wire length and the number of vias are also important goals of global routing.

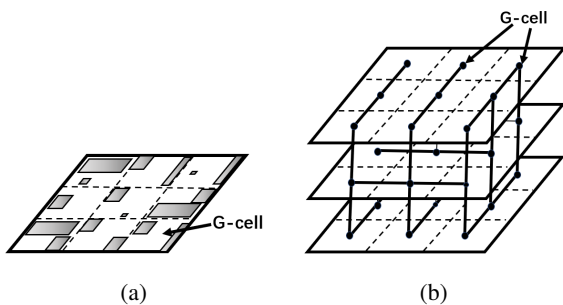


Fig. 1: Grid graph of global routing.

B. Flow of Our Global Router

The flow of our proposed global router contains the initial routing stage and the multi-stage rip-up and rerouting stage, as illustrated in Figure 2. In the initial routing stage, since the solution speed of 3D global routing is slow [6], we first compress the 3D routing area into a 2D area for performing 2D global routing. On the 2D grid graph, we first build a Rectilinear Steiner Minimum Tree (RSMT) for each

net using the tool FLUTE [14], which decomposes each multi-pin net into a set of two-pin nets. Secondly, we establish a pathfinding model based on Integer Linear Programming (ILP) for the 2D global routing problem (Subsection III-A). We use the Lagrangian relaxation method combining with a direction-aware weighted A* algorithm to quickly solve the model to obtain an initial global routing solution (Subsections III-B and III-C).

In the multi-stage rip-up and rerouting stage, we use monotonic routing and maze routing to optimize wire length and overflow of the initial routing result, until the total overflow decreases to zero or the iteration termination condition is reached (Subsection III-D). Finally, layer assignment [15] is performed and the result is restored to 3D grid graph. Details of our global routing techniques are presented in Section III.

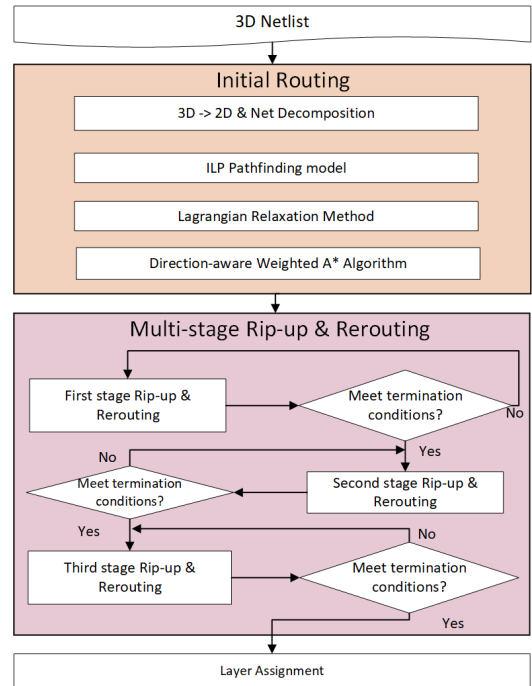


Fig. 2: Design flow of our global router.

III. GLOBAL ROUTING TECHNIQUES

This section introduces core global routing techniques adopted in the initial routing and the rip-up and rerouting stages. Namely, the ILP based pathfinding model and its Lagrangian relaxation, the direction-aware weighted A*-algorithm, and the details of the multi-stage rip-up & rerouting.

A. ILP Based Pathfinding Model

In the previous ILP based global routing models, such as GRIP [6], BonnRoute [7] and Ref. [10], multiple minimum rectilinear Steiner trees need to be generated for each net in advance as routing pattern candidates, which may limit the solution quality. In this subsection, we focus on establishing an ILP based pathfinding model without considering routing patterns prior for global routing. By solving the ILP using the Lagrangian relaxation method, all nets are connected and the total overflow and wire length are minimized.

In our pathfinding model, we need to consider simultaneously the incoming and outgoing edges of each vertex in the grid graph $G(V, E)$. Hence, we construct a directed grid graph $D(V, \mathcal{E})$ corresponding to the undirected grid graph $G(V, E)$. For each undirected edge $e \in E$, we define two directed edges with opposite directions in $\mathcal{E}$, i.e., positive direction edge and negative direction edge, to connect the two end nodes of edge e . As illustrated in Figure 3, a positive direction edge is a

directed edge from the left (bottom) vertex to the right (top) vertex, and a negative edge points in an opposite direction, respectively. In Figure 3(b), the defined negative direction edge $\hat{e}_-$ and positive direction edge $\hat{e}_+$ correspond to the undirected edge e in Figure 3(a). For vertex u , $\hat{e}_-$ is its incoming edge and $\hat{e}_+$ is its outgoing edge. Note that, when the edge $\hat{e}_+$ or $\hat{e}_-$ is used to route a wire, it is considered as the routing edge e is utilized.

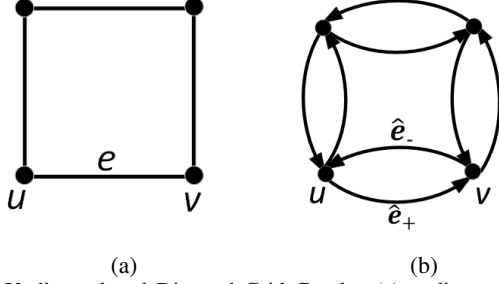


Fig. 3: Undirected and Directed Grid Graphs. (a) undirected edge e . (b) positive direction edge $\hat{e}_+$ and negative direction edge $\hat{e}_-$ corresponding to the undirected edge e .

Table I lists the notations required for the establishment of our ILP based pathfinding model. It should be noted that the initial routing is carried out after all nets are decomposed into two-pin nets n using FLUTE [14].

TABLE I: Symbols and Definitions

Symbols	Definitions
N	The set of nets n
E	The set of routing edges
$\mathcal{E}$	The set of directed edges
V	The vertex set of routing grid graph
e	Undirected routing edge
$\hat{e}$	Directed routing edge
$\hat{e}_+$	Positive direction edge corresponding to the undirected edge e
$\hat{e}_-$	Negative direction edge corresponding to the undirected edge e
$\mathcal{E}_+(v)$	The set of edges outgoing from vertex v
$\mathcal{E}_-(v)$	The set of incoming edges of vertex v
$so(n)$	Source node of net n
$si(n)$	Sink node of net n
$x_{n,\hat{e}}$	Binary variable representing whether the directed edge $\hat{e}$ is used to route net n
$c_{n,e}$	The cost of net n routing edge e
$cap(e)$	Routing capacity of edge e

First of all, a routing result must meet the capacity constraint. That is, the number of wires passing through each routing edge in the undirected grid graph $G(V, E)$ cannot exceed its capacity limit:

$$\sum_{n \in N} (x_{n,\hat{e}_+} + x_{n,\hat{e}_-}) \leq cap(e), \quad \forall e \in E. \quad (1)$$

Second, it is necessary to ensure that there is a connection path for each net. To this end, we divide the nodes in V into three categories: the starting node of the path, the intermediate nodes, and the end node, and consider the constraints on the three types of nodes one by one as follows.

For the starting node of a path in the directed grid graph $D(V, \mathcal{E})$, i.e., the source node of the net, it is necessary to ensure that the source node sends out an edge of the path in $D(V, \mathcal{E})$, while avoiding the formation of a cycle, and ensuring that the source node cannot have an entering edge of the path in $D(V, \mathcal{E})$: for all $n \in N$,

$$\sum_{\hat{e} \in \mathcal{E}_+(so(n))} x_{n,\hat{e}} = 1, \quad (2)$$

$$\sum_{\hat{e} \in \mathcal{E}_-(so(n))} x_{n,\hat{e}} = 0. \quad (3)$$

For the end node of the path in $D(V, \mathcal{E})$, that is, the sink node of the net, it needs to have an entering edge of the path in $D(V, \mathcal{E})$. Similarly, there cannot be a cycle, that is, no edge of the path can be sent out from the sink node: for all $n \in N$,

$$\sum_{\hat{e} \in \mathcal{E}_-(si(n))} x_{n,\hat{e}} = 1, \quad (4)$$

$$\sum_{\hat{e} \in \mathcal{E}_+(si(n))} x_{n,\hat{e}} = 0. \quad (5)$$

For any intermediate node v that is neither source nor sink, in $D(V, \mathcal{E})$ the number of entering edges of the path into v needs to be equal to that of outgoing edges of the path from v . That is, for all $n \in N$,

$$\sum_{\hat{e} \in \mathcal{E}_-(v)} x_{n,\hat{e}} = \sum_{\hat{e} \in \mathcal{E}_+(v)} x_{n,\hat{e}}, \quad \forall v \notin \{so(n), si(n)\}. \quad (6)$$

After ensuring that each net has a connected path in $D(V, \mathcal{E})$ and there is no congestion, we only need to minimize the routing cost. Since the cost of routing the directed edges $\hat{e}_+$ and $\hat{e}_-$ corresponding to the undirected edge e is the same, the objective is further converted equivalently to

$$\min_{x_{n,\hat{e}} \in \{0,1\}} \sum_{n,e} (x_{n,\hat{e}_+} + x_{n,\hat{e}_-}) * c_{n,e} \quad (7)$$

where $c_{n,e}$ is the cost of net n routing e . If it is defined as 1, then the objective function means minimizing the wire length.

Then, our VLSI global routing formulation is minimizing the cost function in Equation (7) subject to the constraints (1)-(6). Note that, the number of vias is not considered explicitly in this formulation, however, it will be handled implicitly in the solution algorithm detailed in Subsection III-C.

B. Lagrangian Relaxation Method and Initial Routing

Lagrangian relaxation method has been used in solving the VLSI global routing problem. BFG-R [11] develops a discrete Lagrange multiplier cost framework for 3D global routing, under which multipliers are updated for guiding rip-up and rerouting directly. Once a net is routed, it immediately updates the multipliers heuristically, so the routing quality is affected by the net order and the router may take a lot of runtime. To avoid this issue and for quicker and better solution, we adopt the Lagrangian relaxation method in 2D space combined with gradient ascent to update the multipliers to solve our pathfinding model and obtain an initial routing solution.

First, by Lagrangian the constraint (1) is relaxed into the objective function (7), which leads to the problem

$$\min_{x_{n,\hat{e}} \in \{0,1\}} \sum_{n,e} c_{n,e} (x_{n,\hat{e}_+} + x_{n,\hat{e}_-}) + \sum_e \mu_e \left(\sum_n (x_{n,\hat{e}_+} + x_{n,\hat{e}_-}) - cap(e) \right) \quad (8)$$

s.t. $(2) - (6), \forall n \in N.$

where $\mu_e \geq 0$ is a Lagrange multiplier.

Since $cap(e)$ is a constant and μ_e is set prior, problem (8) can be simplified to

$$\min_{x_{n,\hat{e}} \in \{0,1\}} \sum_n \sum_e C_{n,e} * (x_{n,\hat{e}_+} + x_{n,\hat{e}_-}) - const \quad (9)$$

s.t. $(2) - (6), \forall n \in N.$

where $C_{n,e}$ and $const$ are both constants related to μ_e , and it is obvious that $C_{n,e}$ is the new cost of routing edge e .

Constraints (2)-(6) ensure that there is a connected path for each net, and the objective function in problem (9) is decomposable for each net.

Therefore, problem (9) can be decomposed equivalently into the sum of the optimal values for each net n of the problem

$$\min_{x_{n,e} \in \{0,1\}} \sum_e C_{n,e} * (x_{n,\hat{e}_+} + x_{n,\hat{e}_-}) - \text{const} \\ \text{s.t.} \quad (2) - (6).$$

Essentially, the above problem is to find a shortest path from the source node to the sink node in $D(V, \mathcal{E})$.

Dijkstra's algorithm is usually used to find a minimum cost path between two nodes. However, its complexity is usually $O((|V| + |E|)\log|V|)$, and it is not easy to consider the number of vias in the algorithm. Moreover, there are a large number of nets to be routed, hence the above problem must be solved more quickly and with considering the number of vias. To this end, we propose an A*-algorithm to solve the above problem, and at the same time consider the need to optimize the number of vias. The algorithm is called direction-aware weighted A*-algorithm *DAWA** and will be presented in Subsection III-C.

Let the optimal value of problem (8) be $f(\mu_e)$, then the dual problem of the original problem (1)-(7) is

$$\max_{\mu_e \geq 0} f(\mu_e). \quad (10)$$

We use the conventional gradient ascent method to solve problem (10). The direction of gradient ascent at the k -th iteration is written as d^k . Let a be the step length, and let *time* be the maximum number of iterations of gradient ascent. Algorithm 1 gives the detailed process.

Algorithm 1: Gradient Ascent Algorithm

Input: $\mu_e^0, C_{n,e}^0(\mu_e^0), a, \text{time}$.

Output: Routing result of two-pin nets $x_{n,\hat{e}}$.

```

1 for  $k = 0$  to time do
2   for  $n \in N$  do
3     Route net  $n$  by DAWA* according to  $C_{n,e}^k(\mu_e^k)$  and
       get  $x_{n,\hat{e}}^k$ ;
4   end
5    $d^k = \nabla f(\mu_e^k)$ ;
6   if  $|\mu_e^k - \mu_e^{k-1}| < \varepsilon$  then
7     return result  $x_{n,\hat{e}}^k$ ;
8   else
9      $\mu_e^{k+1} = \mu_e^k + a \times d^k$ ;
10    if  $\mu_e^{k+1} < 0$  then
11       $\mu_e^{k+1} = 0$ ;
12    end
13    update  $C_{n,e}^{k+1}(\mu_e^{k+1})$ ;
14  end
15 end
16 return result  $x_{n,\hat{e}}$ .
```

The optimal value of problem (10) provides a lower bound of the original problem (1)-(7). If an optimal solution of problem (10) is feasible for problem (1)-(7), i.e., constraints (1)-(6) are satisfied, then it also is an optimal solution of the problem. However, due to the problem characteristics, the solution result may not meet the constraint (1), i.e., the capacity constraint is not satisfied. Therefore, it is necessary to perform rip-up & reroute to obtain a final legal result, which will be provided in Subsection III-D.

C. Direction-aware Weighted A*-Algorithm

In Subsection III-B, we need to find a shortest path in $D(V, \mathcal{E})$ for each net according to the weighted cost of each edge $C_{n,e}$. Traditional routers usually use maze routing or Dijkstra's algorithm, but they are not fast enough for our application. To address this issue, the A*-algorithm

is adopted, which uses the following cost function [16] to calculate the priority of nodes:

$$C_f(n) = C_g(n) + Dis_M(n), \quad (11)$$

where $C_f(n)$ represents the comprehensive cost of node n , $C_g(n)$ represents the cost of node n reaching the source node and $Dis_M(n)$ is the Manhattan distance between node n and the sink.

In global routing, the routing target is not only the wire length but also the number of vias. Therefore, to get a good result in 3D routing, excessive vias (bends) should be avoided during 2D routing. In order to avoid the A*-algorithm from generating too many vias in the minimum cost path, we improve the A*-algorithm by adding the via cost item $Via(n)$, and propose a direction-aware weighted A*-algorithm *DAWA**. If the next node in net n has a bend, we add a via cost to Equation (11):

$$C_f(n) = C_g(n) + Dis_M(n) + Via(n),$$

$$Via(n) = \left\lceil \frac{l}{2} \right\rceil \times C_{base},$$

where l is the number of routing layers, C_{base} is the expected cost of the via, which is an artificially set parameter.

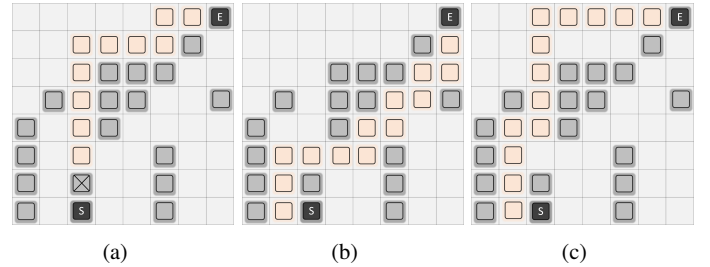


Fig. 4: Vias by different routing algorithms. (a) 3-bend routing. (b) A*-algorithm. (c) Direction-aware weighted A*-algorithm.

Figure 4 compares the routing effect of *DAWA** to some other routing algorithms. The black cells in the figure are the source and end nodes, the grey cells are obstacles or congestion positions, and the others are feasible areas. Suppose all routing edge weights are the same, except that the $C_{n,e}$ from the source cell to its right hand side cell is much higher than that to its left hand side cell. Paths from the source node to the end node by respective algorithms are indicated with light yellow cells in the figure.

The results in Figure 4 show that, the traditional A*-algorithm generates 9 bends (Figure 4(b)) because it is not direction aware. While, *DAWA** has only 4 bends (Figure 4(c)), which can be routed outside the bounding box of the two-pin net compared to monotonic routing. The traditional three-bend routing cannot produce a feasible solution (Figure 4(a)).

D. Multi-stage Rip-up & Rerouting

After initial routing, traditional routers use maze routing to iteratively rip-up and reroute for reducing congestion. Since only one or two stages are adopted, and each stage uses the same routing strategy, this kind of methods may fall into local optimal solution with the increase of iterations [13]. Hence the subsequent iterations of the maze routing spend a lot of runtime to resolve a small amount of congestion.

To handle this issue, we propose a multi-stage rip-up and rerouting algorithm on the undirected grid graph $G(V, E)$. Each stage uses different routing algorithms and cost functions to balance the two goals of minimizing overflow and minimizing wire length. The first stage reduces the wire length without increasing overflow. The second stage reduces the overflow while preventing the wire length from increasing too fast. The third stage divides the nets into routing overflow edges and without routing overflow edges, and optimizes the overflow and wire length respectively.

1st stage: After the initial routing, we get a routing result on the undirected grid graph $G(V, E)$ with less congestion and a slightly longer wire length. Therefore, we first use monotonic routing to optimize the wire length while ensuring that the congestion value does not increase. We use the cost function for each routing edge:

$$cost_1(e) = 1 + \frac{p}{1 + e^{-\varepsilon \times (d(e) - cap(e))}} + c_v(e),$$

where p and ε are user defined parameters, set to 0.8 and 2 respectively, $c_v(e)$ is the cost of a via, which is 3 if routing direction changed. When the overflow reduction in the current iteration is less than 2%, the second stage will be executed.

2nd stage: At this stage, the adaptive multi-source multi-sink maze routing (AMMMR) [17] is used to optimize the overflow value. In order to ensure that the wire length does not increase too much, when the wire length of the AMMMR iteration is greater than 5% of the sum of the half-perimeters of all two-pin nets, or the iteration counts exceed 50, it is terminated. We use the cost function framework in [17] for each routing edge in $G(V, E)$:

$$cost_2(e) = 1 - e^{-\beta e^{-\gamma k}} + c_h(e) \times c_{p_1}(e) + c_v(e),$$

where values of β and γ are constant, k is the current rip-up & reroute iteration count, and $c_h(e)$ is the historical term of the edge, increased by 1 if that edge has overflow. The penalty $c_{p_1}(e)$ is a user-defined parameter.

3rd stage: After the second stage of rip-up & rerouting, most nets route edges in $G(V, E)$ with very few overflow, and some do not. For nets that do not route overflow edges, we use monotonic routing to optimize the wire length as much as possible, while ensuring that overflow does not increase. At this point, we use the cost function for each routing edge in $G(V, E)$:

$$cost_3(e) = \begin{cases} C, & \text{if } d(e) \geq cap(e); \\ 1, & \text{if } d(e) < cap(e), \end{cases} \quad (12)$$

where C is an extremely large constant, which is set to 10000, thus guaranteeing that the number of congested edges does not increase.

For those nets routing congested edges in $G(V, E)$, we first set a forbidden region [13] and use AMMMR to optimize overflow, until the total overflow of the grid graph $G(V, E)$ remains constant (stable) for many iterations or decreases to 0. In this step, we use the cost function for each routing edge as follows:

$$cost_4(e) = 1 - e^{-\beta e^{-\gamma k}} + c_f(e) + c_v(e),$$

$$c_f(e) = \begin{cases} C_p \times (d(e)/cap(e)), & \text{if } e \text{ is in forbidden regions;} \\ c_h(e) \times c_{p_2}(e), & \text{otherwise,} \end{cases}$$

where C_p is user-defined, $c_{p_2}(e)$ is a new penalty item related to congestion in [13].

Finally, the same as dealing with the nets without routing congested edges in $G(V, E)$, we use monotonic routing and cost function (12) to optimize the wire length.

E. Layer Assignment

After the multi-stage rip-up & rerouting, a 2D routing result on $G(V, E)$ is obtained, and we still need to assign it to the 3D space through layer assignment. In this paper, the layer assignment algorithm of dynamic programming [15] is used. This algorithm first determines the allocation order of the nets according to the factors such as wire length and pin numbers, and then uses dynamic programming for each net in turn, to allocate it to an appropriate metal layer.

IV. EXPERIMENTAL RESULTS

We used C++ programming language to implement our global routing algorithm, conduct experiments on the ISPD08 and ISPD18 benchmark circuits [18], [19], on a 64-bit Linux workstation with Intel Core 2.4 GHz CPU and 32 GB memory in single-threaded mode. For

the experimental results, we first present the initial routing results of our global router, and those by pattern routing which is the current popular initial routing algorithm. Then the ISPD08 benchmark is used to demonstrate the performance of our global routing algorithm. Finally, we compare our algorithm with two state-of-the-art works CUGR [4] and SPRoute 2.0 [20] using the ISPD18 benchmarks.

A. Comparison of Initial Routing Results

In this subsection, to demonstrate the effectiveness of our pathfinding model and initial routing algorithm, we compare our initial routing results to those obtained by SPRoute 2.0 which uses pattern routing on the ISPD08 benchmarks n6 and a4.

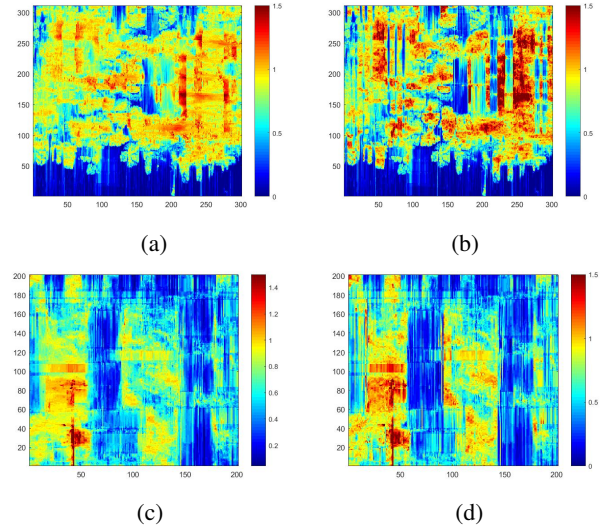


Fig. 5: Initial routing congestion heatmaps. (a) initial routing result of our global router on benchmark n6. (b) initial routing result of SPRoute 2.0 on benchmark n6. (c) initial routing result of our global router on benchmark a4. (d) initial routing result of SPRoute 2.0 on benchmark a4.

Figure 5 gives the partial planar congestion heatmaps of ISPD08 benchmarks n6 and a4 by using our initial routing algorithm and the initial routing of SPRoute 2.0, respectively. Figures 5(a)(c) are the results obtained by our algorithm. The images intuitively show that our algorithm obtains initial routing results with less congestion. This phenomenon indicates that our global router will be dealing with fewer nets in the rip-up and rerouting stage, therefore reducing the runtime.

B. Comparisons on the ISPD08 Benchmarks

In this subsection, we compare our global routing algorithm with SPRoute 1.0 [3] and SPRoute 2.0 [20], which are two state-of-the-art 2D global routers, on the ISPD08 benchmarks. Both of SPRoute 1.0 [3] and SPRoute 2.0 [20] have very similar layer assignment techniques to [15]. To be fair, we run the three global routers on the same machine in single-threaded mode. Note that CUGR [4] does not support the ISPD08 benchmarks, so we cannot compare with it in this subsection. Due to the page limit, we only show the results of the hard-to-route benchmarks in Table II, i.e., no router can make them without overflow.

In the table, “Wire Length” gives the total wire length of each case, “#Via” is the total number of vias, “#Overflow” means the total overflow, and “CPU” is the total runtime in seconds. In terms of wire length and the number of vias, our algorithm outperforms SPRoute 1.0 in all cases, with an average reduction of about 1.1% in wire length and about 1.9% reduction in the number of vias. In terms of average runtime, our algorithm is $3.67\times$ speedup on runtime comparing to SPRoute 1.0. Since SPRoute 2.0 focuses on speed, our algorithm outperforms it in solution quality, but lags behind in speed. Specially, on the benchmark newblue7, our algorithm gets a result with less congestion and shorter wire length in short runtime. This is due to that our algorithm uses multi-stage rip-up & rerouting, which can effectively jump out of local minima.

TABLE II: Comparison with SPRoute 1.0 [3] and SPRoute 2.0 [20] on the ISPD2008 benchmarks

Benchmark	Wire Length			#VIA			#Overflow			CPU (s)		
	SPRoute 1.0	SPRoute 2.0	Ours	SPRoute 1.0	SPRoute 2.0	Ours	SPRoute 1.0	SPRoute 2.0	Ours	SPRoute 1.0	SPRoute 2.0	Ours
bigblue2	4909211	5005825	4787394	4309130	4665945	4192016	6	20651	12	240.54	27.11	230.67
bigblue4	12131018	12900742	11968995	10893146	12572177	10891372	136	134	204	2026.6	248.11	1176.62
newblue1	2434113	2515563	2423492	2260115	2456797	2223437	10	187	6	514.90	90.51	390.86
newblue3	7742056	7923014	7567007	3161130	3627502	3042063	31296	18635	31104	210.58	145.14	1514.23
newblue4	8281132	8644715	8179666	4766135	5418401	4685415	146	12735	188	3752.9	196.23	443.93
newblue7	18521102	19328690	18494415	17012451	18847678	16580347	132	12735	128	14438.12	205.61	2106.92
Normalized	1.011	1.054	1.000	1.019	1.145	1.000	1.002	2.056	1.000	3.670	0.164	1.000

TABLE III: Comparison with SPRoute 2.0 [20] and CUGR [4] on the ISPD2018 benchmarks

Benchmark	Wire Length			#VIA			#Overflow			CPU (s)		
	CUGR	SPRoute 2.0	Ours	CUGR	SPRoute 2.0	Ours	CUGR	SPRoute 2.0	Ours	CUGR	SPRoute 2.0	Ours
ispd18_test1	410518	417181	415116	27231	30460	29677	0	0	0	1.97	0.11	0.14
ispd18_test2	7609620	7709980	7699590	299751	338191	324846	0	0	0	10.49	2.61	1.91
ispd18_test3	8545660	8644190	8626810	300714	336061	326618	0	0	0	12.84	2.51	3.26
ispd18_test4	25996800	26246000	26121900	628426	680322	677125	0	0	0	32.96	6.71	8.91
ispd18_test5	26997000	27307600	27183700	855742	803900	802244	0	0	0	59.16	8.06	12.72
ispd18_test6	34852100	35132200	34980000	1293960	1227804	1219063	0	0	0	56.81	9.22	10.04
ispd18_test7	63804800	64964200	64024700	2109410	1979427	1969051	0	0	0	244.18	22.45	42.73
ispd18_test8	64380000	64964200	64224500	2174530	2109296	2082828	0	0	0	263.81	22.75	54.31
ispd18_test9	53193300	53981500	53674000	2180600	1995119	1987919	0	0	0	84.36	16.63	16.10
ispd18_test10	66778500	71709900	67400100	2308290	2182178	2175317	0	0	0	256.11	34.53	59.19
Normalized	0.995	1.017	1.000	1.051	1.008	1.000	1.000	1.000	1.000	4.891	0.600	1.000

C. Comparisons on the ISPD18 Benchmarks

In this subsection, we compare our global routing algorithm with SPRoute 2.0 [20] and CUGR [4] on the ISPD18 benchmarks [19]. CUGR [4] is a state-of-the-art 3D global router. To be fair, we perform the three global routers on the same machine in single-threaded mode. Table III gives the results of the three routers. Compared to CUGR, our algorithm achieves 5.1% less the average number of vias, and has $4.89\times$ speedup on runtime, while maintaining the average wire length at the same level. In comparison with SPRoute 2.0, the average wire length of our algorithm is reduced by 1.7% and the average number of vias is comparable. In terms of average runtime, since our algorithm solves the ILP based pathfinding model in the initial routing stage to obtain a better solution, it takes more runtime than SPRoute 2.0.

V. CONCLUSION

Based on ILP, this paper has proposed a pathfinding model for VLSI global routing. The Lagrangian of the model is solved by the gradient ascent method to obtain an initial routing quickly, in which the proposed direction-aware A*-algorithm is used. The obtained initial routing result is further improved using a multi-stage rip-up & rerouting. In each stage of rip-up & rerouting, different routing algorithms and cost functions are used to optimize overflow and wire length. Experimental results have shown that our global router achieves high-quality results efficiently on the ISPD08 and ISPD18 benchmarks on the average wire length and the number of vias, compared to state-of-the-art global routers.

REFERENCES

- [1] W.-H. Liu, W.-C. Kao, Y.-L. Li, and K.-Y. Chao, "Nctu-gr 2.0: Multi-threaded collision-aware global routing with bounded-length maze routing," *IEEE Transactions on computer-aided design of integrated circuits and systems*, vol. 32, no. 5, pp. 709–722, 2013.
- [2] Y. Xu, Y. Zhang, and C. Chu, "Fastroute 4.0: Global router with efficient via minimization," in *Proceedings of ACM/IEEE Asia and South Pacific Design Automation Conference*, 2009, pp. 576–581.
- [3] J. He, M. Bartscher, R. Manohar, and K. Pingali, "Sproute: A scalable parallel negotiation-based global router," in *Proceedings of IEEE/ACM International Conference on Computer-Aided Design*, 2019, pp. 1–8.
- [4] J. Liu, C.-W. Pui, F. Wang, and E. F. Young, "Cugr: Detailed-routability-driven 3d global routing with probabilistic resource model," in *Proceedings of ACM/IEEE Design Automation Conference*, 2020, pp. 1–6.
- [5] M. Cho, K. Lu, K. Yuan, and D. Z. Pan, "Boxrouter 2.0: Architecture and implementation of a hybrid and robust global router," in *Proceedings of ACM/IEEE International Conference on Computer-Aided Design*, 2007, pp. 503–508.
- [6] T.-H. Wu, A. Davoodi, and J. T. Linderth, "Grip: Scalable 3d global routing using integer programming," in *Proceedings of ACM/IEEE Design Automation Conference*, 2009, pp. 320–325.
- [7] M. Gester, D. Müller, T. Nieberg, C. Panten, C. Schulte, and J. Vygen, "Algorithms and data structures for fast and good vlsi routing," in *Proceedings of ACM/IEEE Design Automation Conference*, 2012, pp. 459–464.
- [8] Y.-J. Chang, Y.-T. Lee, and T.-C. Wang, "Nthu-route 2.0: A fast and stable global router," in *Proceedings of ACM/IEEE International Conference on Computer-Aided Design*, 2008, pp. 338–343.
- [9] J. Hu, J. A. Roy, and I. L. Markov, "Sidewinder: a scalable ilp-based router," in *Proceedings of ACM International Workshop on System Level Interconnect Prediction*, 2008, pp. 73–80.
- [10] C. Albrecht, "Global routing by new approximation algorithms for multicommodity flow," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 5, pp. 622–632, 2001.
- [11] H. Jin, J. A. Roy, and I. L. Markov, "Completing high-quality global routes," in *Proceedings of IEEE International Symposium on Physical Design*, 2010.
- [12] Y. Xu and C. Chu, "Mgr: Multi-level global router," in *Proceedings of ACM/IEEE International Conference on Computer-Aided Design*, 2011, pp. 250–255.
- [13] H.-Y. Chen, C.-H. Hsu, and Y.-W. Chang, "High-performance global routing with fast overflow reduction," in *Proceedings of ACM/IEEE Asia and South Pacific Design Automation Conference*, 2009, pp. 582–587.
- [14] C. Chu and Y.-C. Wong, "Flute: Fast lookup table based rectilinear steiner minimal tree algorithm for vlsi design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 1, pp. 70–83, 2007.
- [15] T.-H. Lee and T.-C. Wang, "Congestion-constrained layer assignment for via minimization in global routing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 9, pp. 1643–1656, 2008.
- [16] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [17] J.-R. Gao, P.-C. Wu, and T.-C. Wang, "A new global router for modern designs," in *Proceedings of ACM/IEEE Asia and South Pacific Design Automation Conference*, 2008, pp. 232–237.
- [18] ISPD, "Isdpd 2008 global routing contest," <https://www.ispd.cc/contests/08/ispd08rc.html>, 2008.
- [19] S. Dolgov, A. Volkov, L. Wang, and B. Xu, "2019 cad contest: Lef/def based global routing," in *Proceedings of ACM/IEEE International Conference on Computer-Aided Design*, 2019, pp. 1–4.
- [20] J. He, U. Agarwal, Y. Yang, R. Manohar, and K. Pingali, "Sproute 2.0: A detailed-routability-driven deterministic parallel global router with soft capacity," in *Proceedings of ACM/IEEE Asia and South Pacific Design Automation Conference*, 2022, pp. 586–591.